

Task_1.ipynb

Retail Sales Dataset — Data Analysis Notebook

Python · Pandas · Matplotlib · Seaborn

1,000

Transactions

INR 456,000

Total Revenue

3

Categories

9

Columns

#Loading the Libraries and their Alias

```
In [2]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Laod DataSet and Checking Inspection :

```
In [6]: df=pd.read_csv("retail_sales_dataset.csv")
df
```

```
Out [6]:
```

	Transaction ID	Date	Customer ID	Gender	Age	Product Category \
0	1	2023-11-24	CUST001	Male	34	Beauty
1	2	2023-02-27	CUST002	Female	26	Clothing
2	3	2023-01-13	CUST003	Male	50	Electronics
3	4	2023-05-21	CUST004	Male	37	Clothing
4	5	2023-05-06	CUST005	Male	30	Beauty
..	...	...	...	...	...	...
995	996	2023-05-16	CUST996	Male	62	Clothing
996	997	2023-11-17	CUST997	Male	52	Beauty
997	998	2023-10-29	CUST998	Female	23	Beauty
998	999	2023-12-05	CUST999	Female	36	Electronics
999	1000	2023-04-12	CUST1000	Male	47	Electronics

	Quantity	Price per Unit	Total Amount
0	3	50	150
1	2	500	1000
2	1	30	30
3	1	500	500
4	2	50	100
..	...	...	...
995	1	50	50
996	3	30	90
997	4	25	100
998	3	50	150
999	4	30	120

[1000 rows x 9 columns]

```
In [8]: print(df.to_string())
```

```
Out [8]:
```

	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Price p
0	1	2023-11-24	CUST001	Male	34	Beauty	3	50
1	2	2023-02-27	CUST002	Female	26	Clothing	2	500
2	3	2023-01-13	CUST003	Male	50	Electronics	1	30
3	4	2023-05-21	CUST004	Male	37	Clothing	1	500
4	5	2023-05-06	CUST005	Male	30	Beauty	2	50
5	6	2023-04-25	CUST006	Female	45	Beauty	1	30
6	7	2023-03-13	CUST007	Male	46	Clothing	2	25
7	8	2023-02-22	CUST008	Male	30	Electronics	4	25

8	300	9	2023-12-13	CUST009	Male	63	Electronics	2
			600					
9	50	10	2023-10-07	CUST010	Female	52	Clothing	4
			200					
10	50	11	2023-02-14	CUST011	Male	23	Clothing	2
			100					
11	25	12	2023-10-30	CUST012	Male	35	Beauty	3
			75					
12	500	13	2023-08-05	CUST013	Male	22	Electronics	3
			1500					
13	30	14	2023-01-17	CUST014	Male	64	Clothing	4
			120					
14	500	15	2023-01-16	CUST015	Female	42	Electronics	4
			2000					
15	500	16	2023-02-17	CUST016	Male	19	Clothing	3
			1500					
16	25	17	2023-04-22	CUST017	Female	27	Clothing	4
			100					
17	25	18	2023-04-30	CUST018	Female	47	Electronics	2
			50					
18	25	19	2023-09-16	CUST019	Female	62	Clothing	2
			50					
19	300	20	2023-11-05	CUST020	Male	22	Clothing	3
			900					
20	500	21	2023-01-14	CUST021	Female	50	Beauty	1
			500					
21	50	22	2023-10-15	CUST022	Male	18	Clothing	2
			100					
22	30	23	2023-04-12	CUST023	Female	35	Clothing	4
			120					
23	300	24	2023-11-29	CUST024	Female	49	Clothing	1
			300					
24	50	25	2023-12-26	CUST025	Female	64	Beauty	1
			50					
25	500	26	2023-10-07	CUST026	Female	28	Electronics	2
			1000					
26	25	27	2023-08-03	CUST027	Female	38	Beauty	2
			50					
27	500	28	2023-04-23	CUST028	Female	43	Beauty	1
			500					
28	30	29	2023-08-18	CUST029	Female	42	Electronics	1
			30					
29	300	30	2023-10-29	CUST030	Female	39	Beauty	3
			900					
30	300	31	2023-05-23	CUST031	Male	44	Electronics	4
			1200					
31	30	32	2023-01-04	CUST032	Male	30	Beauty	3
			90					
32	50	33	2023-03-23	CUST033	Female	50	Electronics	2
			100					
33	50	34	2023-12-24	CUST034	Female	51	Clothing	3
			150					
... [1001 total lines – first 35 and last 5 shown] ...								
995	50	996	2023-05-16	CUST996	Male	62	Clothing	1
			50					
996	30	997	2023-11-17	CUST997	Male	52	Beauty	3
			90					
997	25	998	2023-10-29	CUST998	Female	23	Beauty	4
			100					
998	50	999	2023-12-05	CUST999	Female	36	Electronics	3
			150					
999	30	1000	2023-04-12	CUST1000	Male	47	Electronics	4
			120					

```
In [12]: print(df.shape)
```

```
Out [12]: (1000, 9)
```

```
In [14]: print(df.isnull())
```

```
Out [14]:
```

	Transaction ID	Date	Customer ID	Gender	Age	Product Category	\
0	False	False	False	False	False	False	
1	False	False	False	False	False	False	
2	False	False	False	False	False	False	
3	False	False	False	False	False	False	
4	False	False	False	False	False	False	
..	...	...	...	...	...	...	
995	False	False	False	False	False	False	
996	False	False	False	False	False	False	
997	False	False	False	False	False	False	
998	False	False	False	False	False	False	
999	False	False	False	False	False	False	

	Quantity	Price per Unit	Total Amount
0	False	False	False
1	False	False	False
2	False	False	False
3	False	False	False
4	False	False	False
..	...	...	...
995	False	False	False
996	False	False	False
997	False	False	False
998	False	False	False
999	False	False	False

[1000 rows x 9 columns]

```
In [16]: print(df.dtypes)
```

```
Out [16]: Transaction ID      int64
Date                      str
Customer ID              str
Gender                   str
Age                      int64
Product Category        str
Quantity                int64
Price per Unit          int64
Total Amount            int64
dtype: object
```

Descriptive Statistics :

1 . Mean of all the numerical columns

```
In [71]: # Performing the operation on the columns " AGE "

import pandas as pd
import statistics as st
df = pd.read_csv("retail_sales_dataset.csv")
mean_age = df["Age"].mean()
```

```
print("Mean of Age :",mean_age)

median_age = (df['Age'].median())
print("Median of Age :",median_age)

mode_age = (df['Age'].mode())
print("Mode of Age :",mode_age)

std_age = (df['Age'].std())
print("Standard Deviation of Age :",std_age)
```

```
Out [71]: Mean of Age : 41.392
Median of Age : 42.0
Mode of Age : 0      43
1      64
Name: Age, dtype: int64
Standard Deviation of Age : 13.681429659122527
```

```
In [77]: # Column - "Transaction ID"

mean_id = df['Transaction ID'].mean()
print("Mean of Transaction_Id :",mean_id)

median_id = (df['Transaction ID'].median())
print("Median of Transaction_Id :",median_id)

mode_id = (df['Transaction ID'].mode())
print("Mode of Transaction_Id :",mode_id)

std_id = (df['Transaction ID'].std())
print("Standard Deviaton of Transaction_Id :",std_id)
```

```
Out [77]: Mean of Transaction_Id : 500.5
Median of Transaction_Id : 500.5
Mode of Transaction_Id : 0      1
1      2
2      3
3      4
4      5
...
995    996
996    997
997    998
998    999
999   1000
Name: Transaction ID, Length: 1000, dtype: int64
Standard Deviaton of Transaction_Id : 288.8194360957494
```

```
In [79]: # Column - "Quantity":

mean_qty = df['Quantity'].mean()
print("Mean of Quantity :",mean_qty)

median_qty = (df['Quantity'].median())
print("Median of Quantity :",median_qty)

mode_qty = (df['Quantity'].mode())
print("Mode of Quantity :",mode_qty)

std_qty = (df['Quantity'].std())
print("Standard Deviaton of Quantity :",std_qty)
```

```
Out [79]: Mean of Quantity : 2.514
Median of Quantity : 3.0
Mode of Quantity : 0    4
Name: Quantity, dtype: int64
Standard Deviaton of Quantity : 1.1327343409145356
```

```
In [81]: # Column - "Price Per Unit" :

mean_price_unit = df['Price per Unit'].mean()
print("Mean of Price per Unit :",mean_price_unit)

median_price_unit = (df['Price per Unit'].median())
print("Median of Price per Unit :",median_price_unit)

mode_price_unit = (df['Price per Unit'].mode())
print("Mode of Price per Unit:",mode_price_unit)

std_price_unit = (df['Price per Unit'].std())
print("Standard Deviaton of Price per Unit :",std_price_unit)
```

```
Out [81]: Mean of Price per Unit : 179.89
Median of Price per Unit : 50.0
Mode of Price per Unit: 0    50
Name: Price per Unit, dtype: int64
Standard Deviaton of Price per Unit : 189.68135627129232
```

```
In [83]: # Column - " Total Amount ":

mean_total_amt = df['Total Amount'].mean()
print("Mean of Total Amount :",mean_total_amt)

median_total_amt = (df['Total Amount'].median())
print("Median of Total Amount :",median_total_amt)

mode_total_amt = (df['Total Amount'].mode())
print("Mode of Total Amount:",mode_total_amt)

std_total_amt = (df['Total Amount'].std())
print("Standard Deviaton of Total Amount :",std_total_amt)
```

```
Out [83]: Mean of Total Amount : 456.0
Median of Total Amount : 135.0
Mode of Total Amount: 0    50
Name: Total Amount, dtype: int64
Standard Deviaton of Total Amount : 559.997631555123
```

Time Series Analysis :

🔪 # Use Line Charts for Ploting Monthly And Quarterly Sales

```
In [85]: # Convert text date column into datetime format

df["Date"] = pd.to_datetime(df["Date"])
print(df["Date"].dtype)
```

```
Out [85]: datetime64[us]
```

```
In [87]: #Finding the Monthly Sales using line charts

monthly_sales = df.resample("ME",on="Date")["Total Amount"].sum()
print(monthly_sales)
```

```
Out [87]: Date
2023-01-31    35450
2023-02-28    44060
2023-03-31    28990
2023-04-30    33870
2023-05-31    53150
2023-06-30    36715
2023-07-31    35465
2023-08-31    36960
2023-09-30    23620
2023-10-31    46580
2023-11-30    34920
2023-12-31    44690
2024-01-31     1530
Freq: ME, Name: Total Amount, dtype: int64
```

```
In [89]: # Monthly Sales Trend Using Plot

import pandas as pd
import matplotlib.pyplot as plt

df["Date"]=pd.to_datetime(df["Date"])
df=df.dropna(subset=["Date","Total Amount"])

monthly_sales = (
    df.resample("ME", on="Date")["Total Amount"]
    .sum()
    .reset_index()
)

plt.figure(figsize=(10,5))

plt.plot(
    monthly_sales["Date"],
    monthly_sales["Total Amount"],
    marker="o",
    color="blue",
    linewidth=0.5
)

plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total_Sales_Amount")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

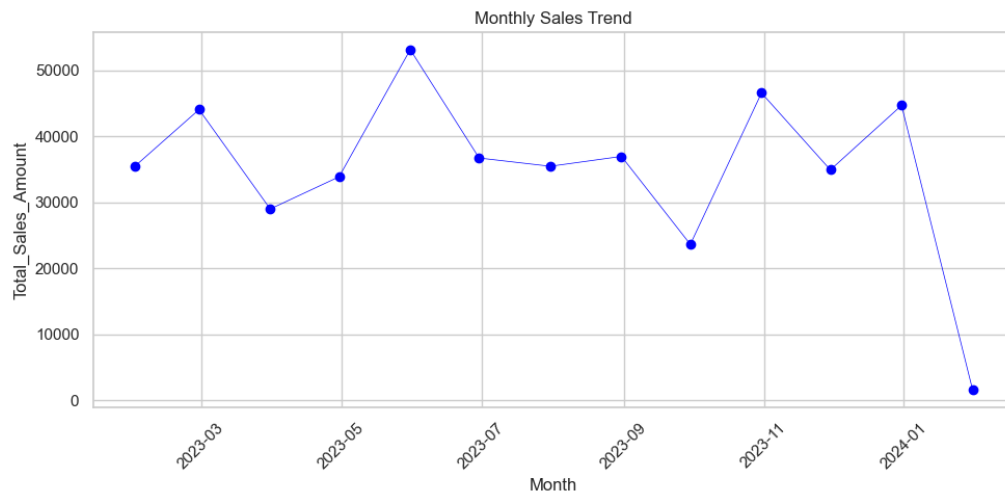


Fig 1 — Monthly Sales Trend (Line Chart)

```
In [91]: #Finding the Quarterly Sales using line charts

Quarterly_Sales_Amount = df.resample("QE",on="Date")["Total Amount"].sum()
print(Quarterly_Sales_Amount)
```

```
Out [91]: Date
2023-03-31    108500
2023-06-30    123735
2023-09-30     96045
2023-12-31    126190
2024-03-31     1530
Freq: QE-DEC, Name: Total Amount, dtype: int64
```

```
In [93]: # Quaterly Sales Trend Plot

df.dropna(subset=["Date","Total Amount"])

Quarterly_Sales_Amount = (
    df.resample("QE",on="Date")["Total Amount"]
    .sum()
    .reset_index()
)

plt.figure(figsize=(8,5))

plt.plot(
    Quarterly_Sales_Amount["Date"],
    Quarterly_Sales_Amount["Total Amount"],
    marker="*",
    color="green",
    linewidth=1
)

plt.title("Quarterly Sales Trend")
plt.xlabel("Quarters")
plt.ylabel("Quarterly_Sales_Amount")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

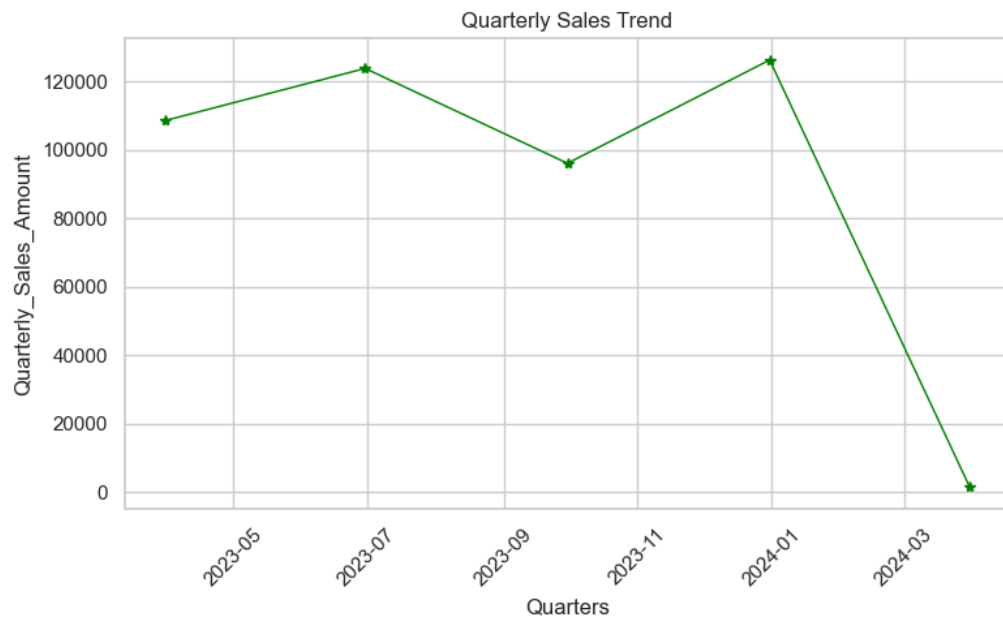



Fig 2 — Quarterly Sales Trend (Line Chart)

Customer Demographics Analysis :

🔗 # Distribution of Customer age groups , Gender breakdown

```
In [95]: print(df.columns)
```

```
Out [95]: Index(['Transaction ID', 'Date', 'Customer ID', 'Gender', 'Age',  
            'Product Category', 'Quantity', 'Price per Unit', 'Total Amount'],  
            dtype='str')
```

```
In [97]: # Customer count by gender
```

```
gender_count = df['Gender'].value_counts()  
print(gender_count)
```

```
Out [97]: Gender  
Female    510  
Male      490  
Name: count, dtype: int64
```

```
In [99]: #Total sales by gender
```

```
Sales_by_gender = df.groupby('Gender')['Total Amount'].sum()  
print(Sales_by_gender)
```

```
Out [99]: Gender  
Female    232840  
Male      223160  
Name: Total Amount, dtype: int64
```

```
In [101]: # Given Two Calculations Used In the Following Bar Charts
```

```
import matplotlib.pyplot as plt
```

```
# Calculate values
sales_by_gender = df.groupby("Gender")["Total Amount"].sum()
gender_count = df["Gender"].value_counts()

# Create 1 row and 2 columns
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# ----- Chart 1: Sales by Gender -----
ax1 = sales_by_gender.plot(
    kind="bar",
    color="skyblue",
    ax=axes[0]
)

ax1.bar_label(
    ax1.containers[0],
    labels=[f"■{value:,.0f}" for value in sales_by_gender.values],
    padding=3,
    fontsize=9
)

ax1.set_title("Total Sales by Gender")
ax1.set_xlabel("Gender")
ax1.set_ylabel("Total Sales")
ax1.tick_params(axis="x", rotation=0)
ax1.grid(axis="y", linestyle="--", alpha=0.5)

# ----- Chart 2: Gender Count -----
ax2 = gender_count.plot(
    kind="bar",
    color="purple",
    ax=axes[1]
)

ax2.bar_label(
    ax2.containers[0],
    labels=[f"{value:,.0f}" for value in gender_count.values],
    padding=3,
    fontsize=9
)

ax2.set_title("Transaction Count by Gender")
ax2.set_xlabel("Gender")
ax2.set_ylabel("Count")
ax2.tick_params(axis="x", rotation=0)
ax2.grid(axis="y", linestyle="--", alpha=0.5)

# Adjust spacing between charts
plt.tight_layout()
plt.show()
```

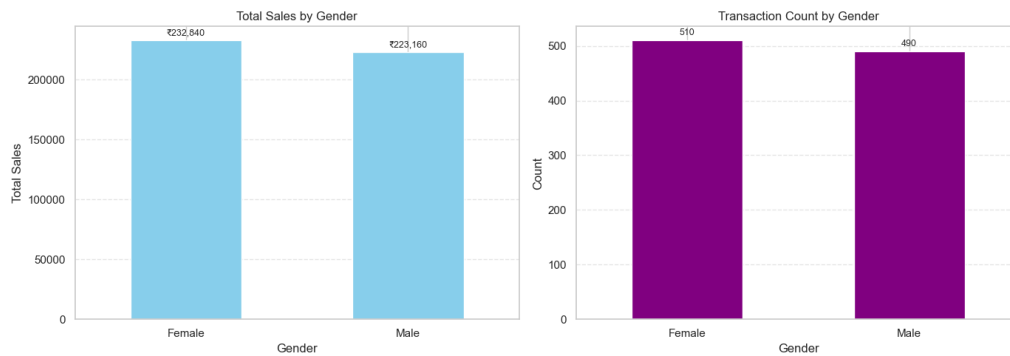


Fig 3 — Total Sales by Gender (left) & Transaction Count by Gender (right)

```
In [103]: #Average sales by gender

Avg_by_gender = df.groupby('Gender')['Total Amount'].mean()
print(Avg_by_gender)
```

```
Out
[103]: Gender
Female    456.549020
Male      455.428571
Name: Total Amount, dtype: float64
```

```
In [105]: # Unique Transaction IDs

total_transactions = df['Transaction ID'].nunique()
print("Total Unique Transaction Id :",total_transactions)
```

```
Out
[105]: Total Unique Transaction Id : 1000
```

```
In [107]: # Total Unique Customers

total_customers = df['Customer ID'].nunique()
print("Total Unique Customers Id :",total_customers)
```

```
Out
[107]: Total Unique Customers Id : 1000
```

age-group distribution summary

```
In [109]: import pandas as pd
import matplotlib.pyplot as plt

# Convert required columns to numeric
df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce")
df["Total Amount"] = pd.to_numeric(df["Total Amount"], errors="coerce")

# Remove rows having missing values in required columns
df = df.dropna(subset=["Age", "Quantity", "Total Amount"])

# Create age groups
bins = [0, 18, 25, 35, 45, 55, 65, 100]

labels = [
```

```

    "Below 18",
    "18-25",
    "26-35",
    "36-45",
    "46-55",
    "56-65",
    "66+"
]

df["Age Group"] = pd.cut(
    df["Age"],
    bins=bins,
    labels=labels,
    include_lowest=True
)

# Age-group summary
age_group_summary = (
    df.groupby("Age Group", observed=False)
    .agg(
        Total_Sales=("Total Amount", "sum"),
        Total_Quantity=("Quantity", "sum"),
        Average_Age=("Age", "mean"),
        Customer_Count=("Customer ID", "nunique"),
        Transaction_Count=("Transaction ID", "nunique")
    )
    .reset_index()
)

print(age_group_summary)

```

Out
[109]:

	Age Group	Total_Sales	Total_Quantity	Average_Age	Customer_Count	\
0	Below 18	11215	54	18.000000	21	
1	18-25	73335	358	21.925676	148	
2	26-35	98480	540	30.473171	205	
3	36-45	91870	506	40.727723	202	
4	46-55	100690	588	50.475983	229	
5	56-65	80410	468	60.184615	195	
6	66+	0	0	NaN	0	

	Transaction_Count
0	21
1	148
2	205
3	202
4	229
5	195
6	0

Product Category Analysis:

In [111]: print(df.columns)

Out
[111]:

```

Index(['Transaction ID', 'Date', 'Customer ID', 'Gender', 'Age',
      'Product Category', 'Quantity', 'Price per Unit', 'Total Amount',
      'Age Group'],
      dtype='str')

```

```
In [113]: # Product Category Count

product_category_count = df['Product Category'].value_counts()
print(product_category_count)
```

```
Out [113]: Product Category
Clothing      351
Electronics   342
Beauty        307
Name: count, dtype: int64
```

```
In [115]: #Total Revenue of Each Product Category

product_category_revenue = df.groupby('Product Category')['Total Amount'].sum()
print(product_category_revenue)
```

```
Out [115]: Product Category
Beauty      143515
Clothing    155580
Electronics 156905
Name: Total Amount, dtype: int64
```

Product Analysis :

```
In [118]: import matplotlib.pyplot as plt

# Calculate values
revenue_by_category = (
    df.groupby("Product Category")["Total Amount"]
      .sum()
      .sort_values(ascending=False)
)

quantity_by_category = (
    df.groupby("Product Category")["Quantity"]
      .sum()
      .sort_values(ascending=False)
)

# Create one output with two charts
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Chart 1: Revenue by category
ax1 = revenue_by_category.plot(
    kind="bar",
    color="orange",
    ax=axes[0]
)

ax1.bar_label(
    ax1.containers[0],
    labels=[f"■{value:,.0f}" for value in revenue_by_category.values],
    padding=3,
    fontsize=9
)

ax1.set_title("Revenue by Product Category")
ax1.set_xlabel("Product Category")
```

```

ax1.set_ylabel("Total Revenue")
ax1.tick_params(axis="x", rotation=0)
ax1.grid(axis="y", linestyle="--", alpha=0.5)

# Chart 2: Quantity by category
ax2 = quantity_by_category.plot(
    kind="bar",
    color="mediumseagreen",
    ax=axes[1]
)

ax2.bar_label(
    ax2.containers[0],
    labels=[f"{value:,.0f}" for value in quantity_by_category.values],
    padding=3,
    fontsize=9
)

ax2.set_title("Best-Selling Category by Quantity")
ax2.set_xlabel("Product Category")
ax2.set_ylabel("Total Quantity Sold")
ax2.tick_params(axis="x", rotation=0)
ax2.grid(axis="y", linestyle="--", alpha=0.5)

fig.suptitle("Product Category Analysis", fontsize=16, fontweight="bold")
plt.tight_layout(rect=[0, 0, 1, 0.93])
plt.show()

```

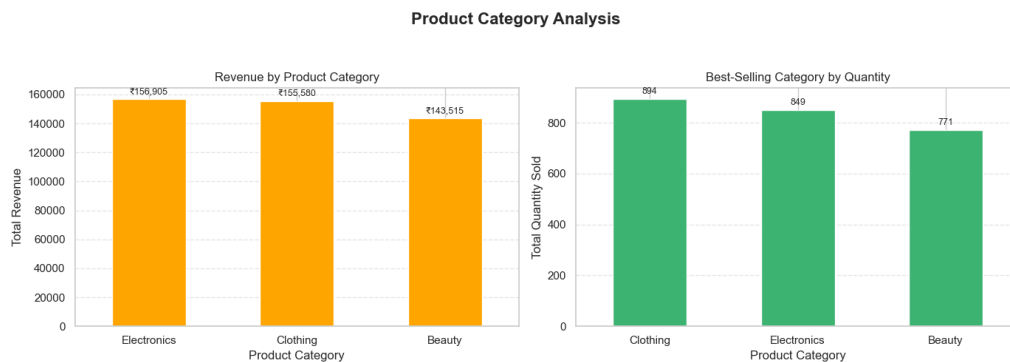


Fig 4 — Revenue by Product Category (left) & Quantity Sold by Category (right)

🔗 # Using Pie Charts Showing the percentages of product category and their total sales percentage

```

In [120]: revenue_by_category = (
            df.groupby("Product Category")["Total Amount"]
            .sum()
            .sort_values(ascending = False)
        )

plt.figure(figsize=(7,5))

plt.pie(
    revenue_by_category.values,
    labels=revenue_by_category.index,
    autopct="%1.1f%%",
    startangle=90,
    colors=["pink", "green", "yellow"],
    wedgeprops={"edgecolor": "white"}
)

```

```
)  
  
plt.title("Revenue Share by Product Category")  
plt.tight_layout()  
plt.show()
```

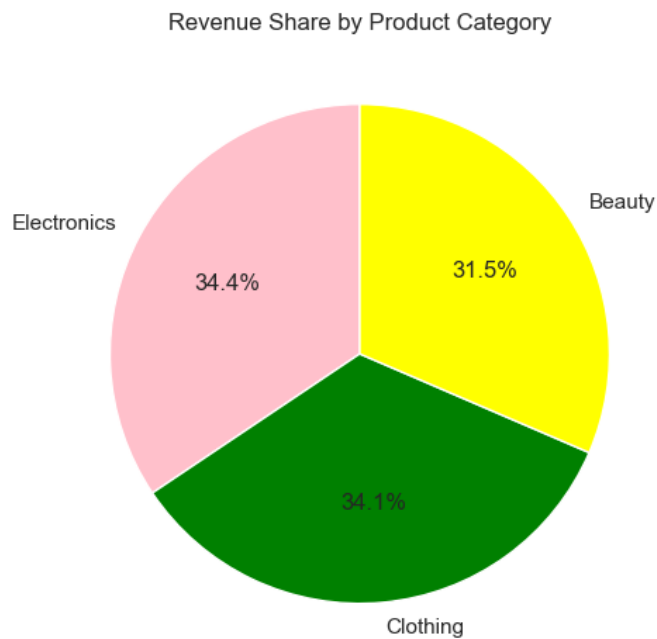


Fig 5 — Revenue Share by Product Category (Pie Chart)

```
In [122]: # Scatter plot: quantity vs total amount  
  
import matplotlib.pyplot as plt  
  
plt.figure(figsize=(8, 5))  
  
plt.scatter(  
    df["Quantity"],  
    df["Total Amount"],  
    color="teal",  
    alpha=0.6,  
    s=60  
)  
  
plt.title("Quantity vs Total Amount")  
plt.xlabel("Quantity Purchased")  
plt.ylabel("Total Amount")  
plt.grid(True, linestyle="--", alpha=0.5)  
plt.tight_layout()  
plt.show()
```

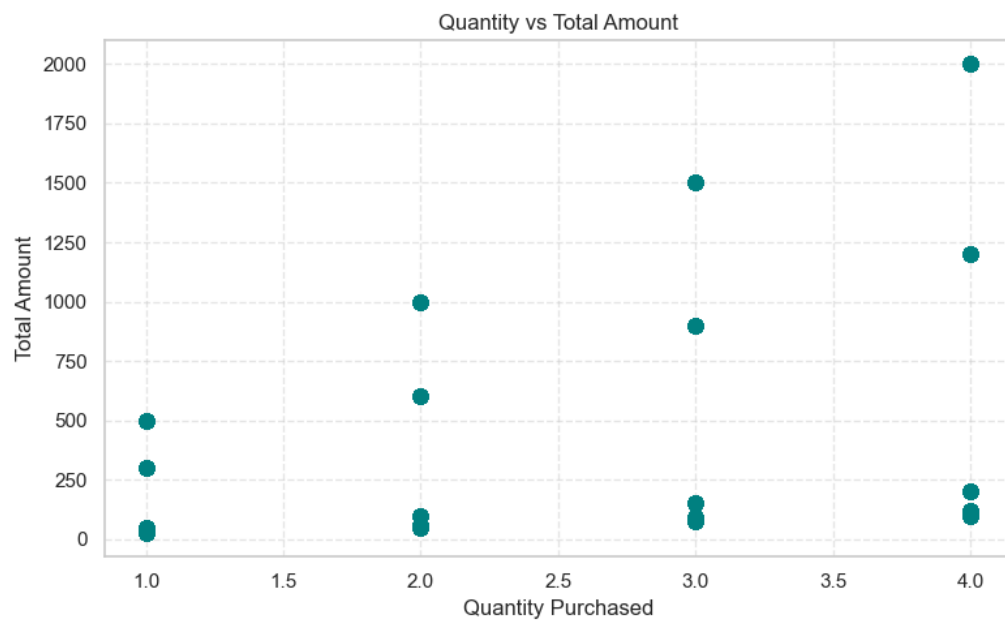


Fig 6 — Scatter Plot: Quantity vs Total Amount

```
In [60]: # Scatter plot by gender

import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="Quantity",
    y="Total Amount",
    hue="Gender",
    s=70,
    alpha=0.7
)

plt.title("Quantity vs Total Amount by Gender")
plt.xlabel("Quantity Purchased")
plt.ylabel("Total Amount")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```

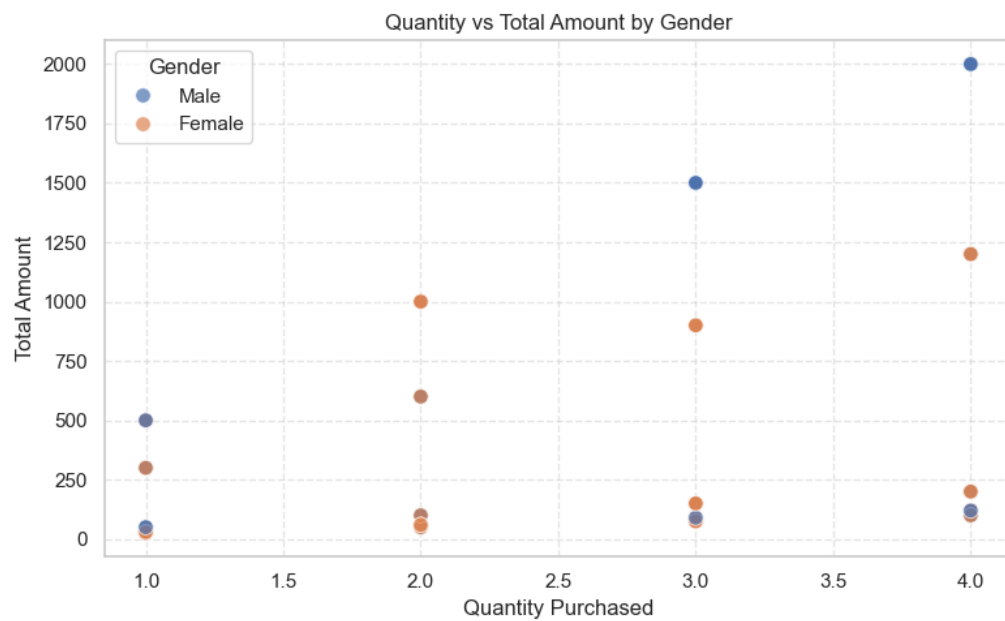



Fig 7 — Scatter Plot: Quantity vs Total Amount (by Gender)

```
In [124]: # Transaction Distribution By Gender

import matplotlib.pyplot as plt

# Count transactions for each gender
gender_count = df["Gender"].value_counts()

# Calculate total transactions
total_transactions = gender_count.sum()

plt.figure(figsize=(7, 6))

plt.pie(
    gender_count.values,
    labels=gender_count.index,
    autopct="%1.1f%%",
    startangle=90,
    colors=["#9370DB", "#FFB6C1", "#87CEEB"],
    wedgeprops={
        "width": 0.45,      # Creates donut hole
        "edgecolor": "white"
    }
)

# Add total transaction count in the center
plt.text(
    0,
    0,
    f"Total\n{total_transactions:,.0f}",
    ha="center",
    va="center",
    fontsize=14,
    fontweight="bold"
)

plt.title("Transaction Distribution by Gender")
plt.tight_layout()
plt.show()
```

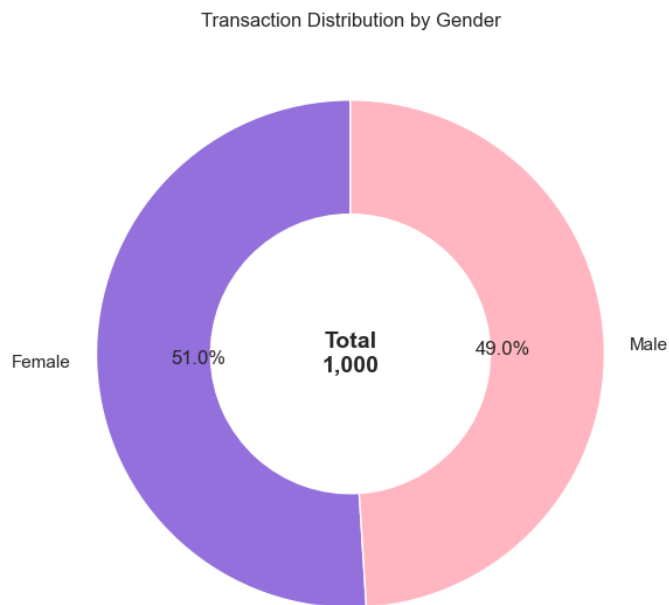


Fig 8 — Transaction Distribution by Gender (Donut Chart)

Correlaton Matrix : Heatmap

```
In [126]: # Find the correlation between the dataset
numeric_columns = df[[
    "Age",
    "Quantity",
    "Price per Unit",
    "Total Amount"
]]

numeric_corr_matrix = numeric_columns.corr()
print(numeric_corr_matrix)
```

```
Out [126]:
```

	Age	Quantity	Price per Unit	Total Amount
Age	1.000000	-0.023737	-0.038423	-0.060568
Quantity	-0.023737	1.000000	0.017501	0.373707
Price per Unit	-0.038423	0.017501	1.000000	0.851925
Total Amount	-0.060568	0.373707	0.851925	1.000000

```
In [128]: # Correlation Heatmap:
import matplotlib.pyplot as plt
import seaborn as sns

numeric_columns = df[[
    "Age",
    "Quantity",
    "Price per Unit",
    "Total Amount"
]]

numeric_corr_matrix = numeric_columns.corr()

plt.figure(figsize=(7,5))
```

```
sns.heatmap(
    numeric_corr_matrix,
    annot = True,
    cmap = "coolwarm",
    fmt=".2f",
    linewidths=0.5,
    vmin=-1,
    vmax=1
)

plt.title("Correlation Heatmap of Sales Dataset")
plt.tight_layout()
plt.show()
```

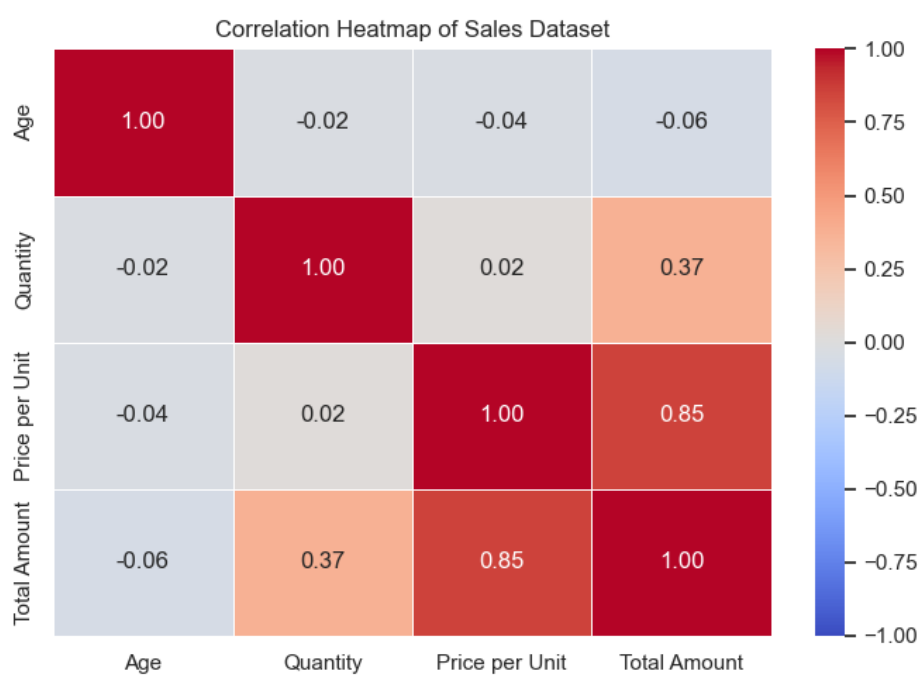


Fig 9 — Correlation Heatmap of Sales Dataset